

## 4. Integrate Git with Docker for source-controlled application builds

Step-1: Create a Dockerfile using **nano** in your project directory.

—> **Dockerfile**

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
# Stage -2
FROM node:18-slim
WORKDIR /app
COPY --from=build /app /app
EXPOSE 5000
CMD ["node", "index.js"]
```

Step-2: Create a index.js file and write in it

—>**index.js**

```
const express = require("express");
const app = express();
const port=3000

app.get("/", (req, res) => {
  res.send("Hello from Git + Docker Integration");
});

app.listen(port, () => {
  console.log('Server is Running at http://localhost:${port}');
});
```

Step-3 : Make sure the **Git** is installed, else install it from

<https://github.com/git-guides/install-git>

Step-4: Run the command :

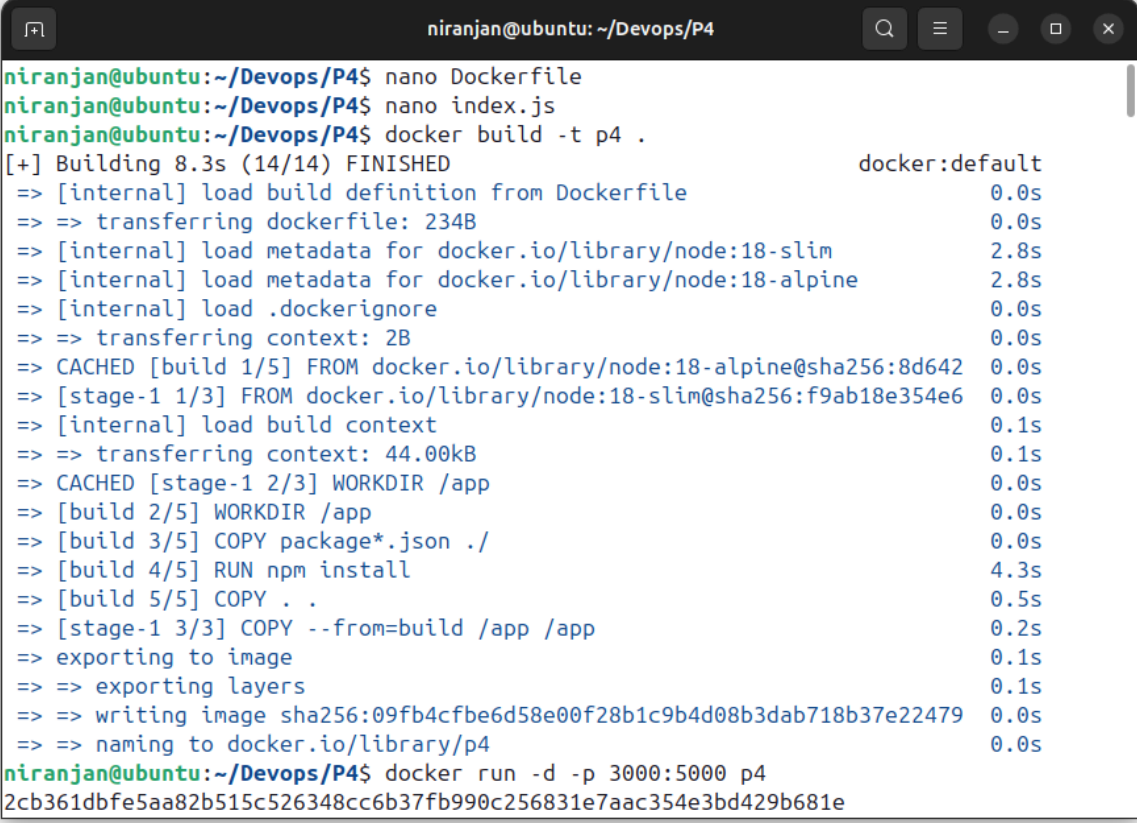
```
$ npm init -y  
$ npm install express  
$ npm install node:18-alpine
```

Step-5: Build the docker image using the below command

```
docker build -t p4 .
```

Step-6: Run the Docker Run command to create a container specifying the port number and name for the container

```
sudo docker run -p 5000:3000 p4
```

A terminal window titled 'niranjan@ubuntu: ~/Devops/P4' showing the execution of Docker commands. The user runs 'nano Dockerfile', 'nano index.js', and 'docker build -t p4 .'. The build process is detailed with progress bars and timestamps, showing the use of cached images and the installation of npm. Finally, the user runs 'docker run -d -p 3000:5000 p4', which returns a long container ID.

```
niranjan@ubuntu:~/Devops/P4$ nano Dockerfile  
niranjan@ubuntu:~/Devops/P4$ nano index.js  
niranjan@ubuntu:~/Devops/P4$ docker build -t p4 .  
[+] Building 8.3s (14/14) FINISHED                                docker:default  
=> [internal] load build definition from Dockerfile                0.0s  
=> => transferring dockerfile: 234B                                0.0s  
=> [internal] load metadata for docker.io/library/node:18-slim     2.8s  
=> [internal] load metadata for docker.io/library/node:18-alpine   2.8s  
=> [internal] load .dockerignore                                   0.0s  
=> => transferring context: 2B                                       0.0s  
=> CACHED [build 1/5] FROM docker.io/library/node:18-alpine@sha256:8d642 0.0s  
=> [stage-1 1/3] FROM docker.io/library/node:18-slim@sha256:f9ab18e354e6 0.0s  
=> [internal] load build context                                   0.1s  
=> => transferring context: 44.00kB                                  0.1s  
=> CACHED [stage-1 2/3] WORKDIR /app                               0.0s  
=> [build 2/5] WORKDIR /app                                         0.0s  
=> [build 3/5] COPY package*.json ./                               0.0s  
=> [build 4/5] RUN npm install                                     4.3s  
=> [build 5/5] COPY . .                                            0.5s  
=> [stage-1 3/3] COPY --from=build /app /app                       0.2s  
=> exporting to image                                              0.1s  
=> => exporting layers                                              0.1s  
=> => writing image sha256:09fb4cfbe6d58e00f28b1c9b4d08b3dab718b37e22479 0.0s  
=> => naming to docker.io/library/p4                                0.0s  
niranjan@ubuntu:~/Devops/P4$ docker run -d -p 3000:5000 p4  
2cb361dbfe5aa82b515c526348cc6b37fb990c256831e7aac354e3bd429b681e
```

Step-7: Test the container by verifying whether its showing some results on <http://localhost:5000> stating **“Hello from Git + Docker Integration”**

You can also check the status of the container by using below command

`docker ps` // displays current running containers

`Docker ps -a` // displays all present containers

Step-8: Initialise your git repository and push all the files.

```
$ git init
```

```
$ git add .
```

```
$ git commit -m "first commit"
```

```
$ git branch -M main
```

```
$ git remote add origin https://github.com/niranjan-achar/program-4.git
```

```
$ git push -u origin main
```

Now, it asks you to enter username and PAT

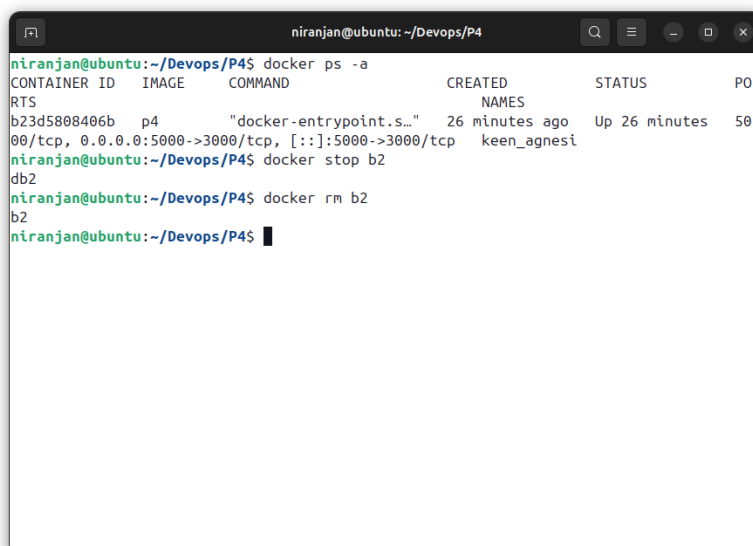
Step-9: Stop the container, remove the container using container number

```
sudo docker container ls -a or docker ps -a
```

```
sudo docker container stop <container-id>
```

```
sudo docker container rm <container-id>
```

```
sudo docker image rm <image-id> or docker rmi <image-id>
```



```
niranjan@ubuntu: ~/Devops/P4
niranjan@ubuntu:~/Devops/P4$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
RTS
b23d5808406b   p4        "docker-entrypoint.s..." 26 minutes ago Up 26 minutes 5000/tcp, 0.0.0.0:5000->3000/tcp, [::]:5000->3000/tcp   keen_agnesi
niranjan@ubuntu:~/Devops/P4$ docker stop b2
b2
niranjan@ubuntu:~/Devops/P4$ docker rm b2
b2
niranjan@ubuntu:~/Devops/P4$
```